STUDENT GUIDE

AI-Assisted Cyber Operations and Authorized Testing

What an assessor needs to know before trusting a model's account of a target

Session	AI + Cyber: AI Powered Cybersecurity, Session 4 of 5
Date	Friday, November 13, 2026
Author	Frazier A. Smith, Ed.D., Central Piedmont Community College
Companions	Participant Packet, Lab Manual, Session 4 Assessment Workbook
Version	1.0, November 13, 2026

How this guide differs from the packet. The Participant Packet tells you what to do this afternoon. This guide explains the ideas behind each step, with a numbered reference for every outside fact, so that you can hand it to a colleague or use it to build a lab of your own. Northbridge Technical College, its staging network, and its people are fictional. The standards, studies, the GTG-1002 case, and CVE-2021-41773 are real and dated. This guide does not discuss the Lab 7 captures, so it is safe to read before the session.

This material is based upon work supported by the U.S. National Science Foundation under Grant No. 2300188. Any opinions, findings, and conclusions or recommendations expressed in this material are those of the author(s) and do not necessarily reflect the views of the National Science Foundation.

START HERE

How to Use This Guide

Session 3 asked whether an analyst can trust a model's account of a threat. Session 4 asks whether an assessor can trust a model's account of a target, and where a human approval has to sit once a model can act as well as advise. The answer depends on three habits that predate AI: work only inside a written authorization, record what a tool printed before anyone interprets it, and confirm a claim with evidence the claim did not supply. A model makes each habit more important, because it produces interpretations faster than an assessor can check them.

Part	Title	Read it when
1	Authorization Is a Professional Competency	You are writing or reading rules of engagement, or deciding whether an action is in scope
2	Raw Output and the Limits of Version Detection	A scan names a product and a version, and someone wants to act on it
3	Hypotheses, Validation, and the Approval Gate	A model has proposed explanations or next steps, and you must decide which to test and how
4	Agents With Tools	Someone proposes letting a model run commands, or you are explaining why that needs a gate
5	What the Measurements Say	You need evidence about how capable current models are at security tasks
6	Purple-Team Translation and Timeline Reconciliation	You must turn a finding into detection and remediation, or two records of one event disagree
7	Redesigning Legacy Labs	You are building Lab 8, or revising a scanning lab of your own

The Argument in One Paragraph

Assessment tools report what a target says about itself, and a model reports what the tool output suggests. Neither report is the target. A version string in a scan is a claim the service made, and a scanner that reads the same string has not confirmed it. An alert is a record that a signature matched, not a record that an attack succeeded. A model asked about either will often answer with a confident, specific, plausible story, and a model with tools will act on that story unless something stops it. The discipline that answers this is old and documented: written rules of engagement, a raw-output record, ranked hypotheses tested against independent evidence, a written approval before any consequential test, and a finding that ends in detection, remediation, and validation. GTG-1002 shows an attacker building the same approval gates into an AI-driven intrusion. The difference between that campaign and an authorized assessment is the authorization.

Conventions

Numbers in brackets are references, listed at the end with URLs. Every reference was opened on November 13, 2026. Dates are publication dates unless the text names another kind of date, such as a revision date or a date added to a catalog. Where this guide refers to an earlier session's Student Guide, it names the session and part.

> Lab 7 is built to be worked, not read about. This guide teaches the checks. It does not say what the Lab 7 captures show.

PART 1

Authorization Is a Professional Competency

The difference between an assessment and an attack is not the tool, the command, or the skill. It is a written authorization from someone who owns the risk, and the assessor's discipline in staying inside it. NIST Special Publication 800-115 defines **rules of engagement (ROE)** as "detailed guidelines and constraints regarding the execution of information security testing," established before the test starts, which give the test team "authority to conduct defined activities without the need for additional permissions" [1]. The definition cuts both ways. Activities the ROE defines need no further permission. Activities it does not define have none.

What the Document Must Settle

SP 800-115 Appendix B is a template with sections for purpose, scope, assumptions and limitations, risks, personnel, the test schedule, and the hours testing may run [1]. The prework for this session reduced the working content to seven elements and an approval rule, and Lab 7 uses the same structure.

Element	What it settles	A common gap
Target	Which hosts, addresses, ports, and names are authorized, and which named neighbors are not	A subnet named where two hosts were meant
Time window	When work may run, with the time zone stated	A window with no time zone
Allowed techniques	What the tester may do, with which tool, from which source	"Standard testing" with no list
Prohibited actions	What nobody may do, with or without approval	Silence on hosted AI services
Data rules	The tier of the data the test touches, where it may go, and how a finding may describe it	No rule for files the tester finds
Stop conditions	Observable events that end the work at once	Conditions no tester could observe
Reporting path	Who receives findings, stops, and notifications, and when	Findings sent to whoever asked
Approval rule	Which tests wait for written approval, from whom, and what a request contains	Approval by whoever is online

Deviations Need Written Permission

SP 800-115 is direct about the document's force: "The plan or ROE should be followed unless specific permission to deviate has been obtained, normally in writing, from the original signatory or individual in command." It recommends that assessors review the ROE periodically during the assessment, "particularly in the case of activities in the target vulnerability validation category" [1]. The same section covers the case Lab 7's rules call immediate notification. If the presence of an adversary is found during the assessment, "it should immediately be reported to the appropriate individual," and assessors should stop assessing the systems involved while the organization responds [1].

The Scope Read-Back

A **scope read-back** is the team's restatement of each element in its own words, signed before any work begins. It turns a document the team received into a commitment the team made. It also exposes disagreement early. When three people write three different sentences for the time window, the window was ambiguous, and the fix costs a chat message instead of an incident. Lab 7 scores the read-back first because every later decision depends on it.

Neighbors, Production Twins, and Someone Else's Scan

The Out-of-Scope Neighbor

Authorized targets rarely sit alone. A staging host shares a subnet with other systems, a test site shares a domain with production, and a lab virtual machine shares a switch with the instructor's console. A well-written target element names the neighbors that are out of scope as well as the targets that are in. The reason is practical. A tool pointed at a range touches every host in it, and a tool pointed at a name may resolve to an address the author of the ROE never considered. Two habits keep a test inside the line: point every command at an address or name copied from the target element, and treat any result that mentions anything else as a stop condition.

The Production Twin

Staging environments are built to match production. That is their purpose, and it changes the stakes of a test. A configuration mistake in staging is usually the mistake production will inherit. Credentials in staging are often real credentials for real services. A finding in staging is therefore often a finding about production before it exists, which is why the Northbridge decision in Lab 7 is a go or no-go for a pilot. It also means a staging test can cause production harm, through shared credentials, shared databases, or shared neighbors.

A Capture You Did Not Authorize

Assessors often work from someone else's output: a scanner export, a capture a colleague forwarded, a log a vendor sent. Each must pass the same test as the assessor's own commands. Was it taken inside the window, against the authorized target, with an allowed technique, from the authorized source? Output that fails any of those tests may still be accurate, and it may still matter. It is not evidence for this assessment. A good observation log records the file, the decision, and the element that decides it, so a reader can see the file was considered and set aside for a reason.

Worked Example: The Fernwood Read-Back

The prework's Fernwood County Library District authorization covered two servers, 198.51.100.10 and 198.51.100.11, and the test site `test.fernwood-library.example`, from 6:00 to 10:00 p.m. on Tuesday, March 9. Three of its proposed actions show the elements at work.

Proposed action	Verdict	Deciding element and reason
Rerun the service enumeration of 198.51.100.10 at 2:00 p.m. on March 10 because the first run was interrupted	Out of scope	Time window. The target and technique are authorized, and the time is not. A second window needs a new authorization
Repeat the page-access check against the production catalog to compare	Out of scope	Prohibited actions. A sound reason to compare does not create authorization. The comparison becomes a recommendation in the report
Confirm an input-handling flaw by submitting a form value that creates a record	Requires approval	Approval required. The target is in scope, and the test changes data

The pattern generalizes. A verdict names one element, and a good reason explains why the other elements do not rescue the action.

PART 2

Raw Output and the Limits of Version Detection

Raw output is what a tool printed, unedited, with the information a reader needs to reproduce it: tool and version, exact command, target, and start time with its time zone. It is the evidence. Everything said about it afterward, by a person or a model, is interpretation, and an interpretation can be checked only against a record of what it interprets.

What `-sT` and `-sV` Do

Nmap's TCP connect scan, `-sT`, asks the operating system to open a full connection to each port with the same connect system call a web browser uses. The Nmap Reference Guide notes that this "completes connections to open target ports" rather than performing the half-open reset of a SYN scan, so it takes longer, sends more packets, and makes target machines "more likely to log the connection" [2]. For an authorized assessment that is a feature. The target's logs and sensors record the test, and the record can be reconciled with the test plan.

Version detection, `-sV`, interrogates open ports after they are found. Nmap's `nmap-service-probes` database "contains probes for querying various services and match expressions to recognize and parse responses," and Nmap tries to determine the protocol, the application name, the version number, and more. The guide adds a limit in the same paragraph: "most services don't provide all of this information" [3]. It also warns against the simpler lookup that version detection replaces. A port number suggests a service, "however, you should not bet your security on this! People can and do run services on strange ports" [3].

A Banner Is a Claim

Version detection matches what a service sends against stored patterns. When a service sends a header or a greeting that names a product and a version, the match reports that product and version. The service wrote that string, and nothing in the protocol requires it to be true. SP 800-115 says so directly: version scanning "is not foolproof, because a security-conscious administrator can alter the transmitted banners or other characteristics in hopes of concealing the service's true nature." It also says version scanning "is far more accurate than simply relying on a scanner's services file" [1]. Both statements hold. A banner is better evidence than a port number, and it is still a claim.

Banners go wrong for ordinary reasons as well as deliberate ones. A reverse proxy or load balancer can answer for the application behind it. A framework can set a default `Server` header that names software it is not. An administrator can copy a header from an old configuration. In each case, the scan line is an accurate record of what the service sent and an inaccurate description of what the service is.

Evidence a Banner Does Not Supply

Evidence	What it can show
Protocol behavior	How the service answers requests the banner does not script: the body of an error page, the headers present and their order, the response to a path that does not exist. Different server software answers these differently
Application endpoints	What the application reports about itself on an allowed path, such as a version or health endpoint
Package inventory	What software is installed on the host, from the platform team's records rather than from the network
Telemetry	How the service responded to earlier requests, as recorded by a sensor or a log the service did not write

Each of these rests on a different observation from the banner. Two of them agreeing with each other and disagreeing with the banner is a strong result.

CVE-2021-41773: Why One Version String Draws Traffic

A version string matters because defenders and attackers both map versions to vulnerabilities. CVE-2021-41773 is the clearest recent example of a single version number that names a critical, exploited flaw, and it is the real-world record Lab 7 uses.

Fact	Value and source
Flaw	"A flaw was found in a change made to path normalization in Apache HTTP Server 2.4.49. An attacker could use a path traversal attack to map URLs to files outside the directories configured by Alias-like directives." If CGI scripts are also enabled for the aliased paths, the flaw "could allow for remote code execution" [5, 4]
Affected versions	"This issue only affects Apache 2.4.49 and not earlier versions" [5]
Fixed	Apache HTTP Server 2.4.50, released October 4, 2021 [4]
Incomplete fix	The 2.4.50 fix "was insufficient." The remaining flaw is CVE-2021-42013, which affects 2.4.49 and 2.4.50 and is fixed in 2.4.51, released October 7, 2021 [4, 6]
Severity	NVD published CVE-2021-41773 on October 5, 2021, with a CVSS 3.1 base score of 9.8, Critical, vector AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H, weakness CWE-22 [5]. NIST also scores CVE-2021-42013 at 9.8 [6]
Exploitation	Apache's own entry states, "This issue is known to be exploited in the wild" [4]. CISA added both CVEs to the Known Exploited Vulnerabilities (KEV) catalog on November 3, 2021, with an action due date of November 17, 2021, under the name "Apache HTTP Server Path Traversal Vulnerability," and marks known ransomware campaign use for both as "Known" [7]

What the Record Implies for an Assessor

Because the flaw is tied to exactly one release, the version string alone carries the vulnerability claim. A host that reports Apache 2.4.49 is, by that report, a host with a critical exploited flaw, and a vulnerability scanner that reads the same report will list the CVE. That is the efficiency of version-based detection and its weakness. The claim is only as good as the string.

The same specificity makes such a banner a target. Anyone scanning for the flaw can find the host by its version string, whether or not the software behind the string is vulnerable. Part 3 is about telling a claim from a confirmed fact.

The Raw-Output Record

The observation log in Lab 7 records six things per capture: tool and version, command, target as authorized, start time and zone, file name with SHA-256 hash, and what the tool printed. The hash lets a reader confirm the file is the one you read. The target column uses the ROE's wording, so any output whose target does not match is visible at a glance. The Observed column holds only printed output. Writing "Apache 2.4.49" in Observed is correct when the scan printed it. Writing "vulnerable to CVE-2021-41773" there is not, because no tool printed it.

PART 3

Hypotheses, Validation, and the Approval Gate

A model is useful to an assessor in one job above all: proposing explanations for raw output and the tests that would separate them. It is harmful in another: stating one explanation as fact. The four-part request in Lab 7 is built to get the first job and refuse the second.

Ranked Hypotheses

A **hypothesis** is one explanation of the raw output, stated so that evidence could confirm or rule it out. Asking for two to four, most likely first, forces the model to name alternatives. Asking for the evidence that would confirm and rule out each one turns the answer into a test plan. Asking for the least invasive next step, marked against the scope, turns the test plan into something the scope officer can check line by line. The answer is still a proposal. The assessor records which model produced it and when, and the scope officer decides what may run.

Independent Evidence

Two pieces of evidence are **independent** when each rests on its own observation. A scan line and a vulnerability scanner's finding that both read the same banner rest on one observation, the banner. SP 800-115 makes the point about tools in general. Assessors may validate findings "by using a second automated tool and comparing the results," but the comparison tools often produce similar results, including the same false positives, and "manual examination typically provides more accurate results than comparing results from multiple tools" [1]. It also notes that vulnerability scanners "can have a high false positive error rate (i.e., reporting vulnerabilities when none exist)" and that their results need interpretation by someone with networking and operating system expertise [1].

The working rule for Lab 7 follows. Before accepting a service, version, or vulnerability claim, find at least two pieces of evidence that the claim itself did not supply. Then count origins, not documents, which is the Session 3 rule applied to tool output.

Consequential Tests and the Least Invasive Method

A **consequential test** is one whose effects reach beyond reading: it changes data, can disrupt a service, uses a credential, or exposes information the data rules restrict. An **approval gate** holds such a test until a named person who owns the risk approves it in writing. The **least invasive method** demonstrates the finding with the smallest effect. Showing that a file is exposed does not require opening it, and showing that a request reaches a database does not require changing a production record.

The four-part approval request makes the approver's decision possible. It states the finding so far with a pointer to the raw output, the proposed test and its expected effect, a less invasive alternative with a judgment of whether it suffices, and how the target returns to its prior state. A request that omits the alternative asks the approver to trust that none exists. A request that omits the return path asks the approver to accept a change nobody plans to undo.

Confidence Statements

ICD 203 requires analytic products to express likelihood with fixed terms, from "almost no chance" (01 to 05 percent) to "almost certain" (95 to 99 percent), and to explain both "the likelihood of occurrence of an event or development, and the analyst's confidence in the basis for this judgment" [8]. An assessment finding has more than one part, and each part can deserve a different confidence. A team can hold high confidence that a file was exposed, because a capture shows it, and low confidence about who retrieved it, because one address in one log is all the evidence there is. The finding is stronger for saying so.

A Worked Example: The Fernwood Search Error

The prework used a synthetic log export from the Fernwood County Library District's test site. The whole chain from Parts 2 and 3 fits on it, and none of it concerns Northbridge.

Raw Output

```
19:02:11  198.51.100.10  GET /search?q=history    200  1.2 KB
19:02:30  198.51.100.10  GET /search?q=o%27brien  500  0.3 KB
19:02:45  198.51.100.10  GET /search?q=obrien     200  1.1 KB
```

The observation is narrow. A search containing an apostrophe, encoded as `%27`, returned a server error, and the same search without the apostrophe succeeded. The log does not say why. "The search is vulnerable to injection" is an interpretation, and it belongs in the hypothesis record.

Ranked Hypotheses

#	Hypothesis	Evidence to confirm	Evidence to rule out	Least invasive next step and scope verdict
1	The search builds a database query from the input without escaping it	A query syntax error in the server's error log at 19:02:30	No database error, or an error in an unrelated component	Read the server error log for 19:02:30. In scope, a review of provided logs
2	Input validation rejects the character and the application handles the rejection badly	An application exception naming validation, with no query text	A database error naming the query	The same log review
3	The error was a coincidence, such as a timeout	Other 500 responses at the same time on unrelated paths	The error repeats on a second apostrophe search and not on others	Repeat one read-only apostrophe search with the test account. In scope as authenticated testing, inside the window

The next step that would confirm hypothesis 1 beyond doubt, submitting a value that creates a record, changes data. Under the Fernwood authorization it requires approval, and the Participant Packet shows the four-part request for it.

Purple-Team Translation

The prework carried the finding through translation, after a server error log showed a query syntax error at that time.

Detection idea	Alert when the web-server log shows a burst of 500 responses on the search path from one client. Expect occasional false positives from patrons who search for names with apostrophes.
Remediation	The application owner changes the search to use a parameterized query, so input never becomes part of the query text.
Validation step	Repeat the original search after the change. The expected result is status 200 with ordinary results, and no new entry in the error log.
Confidence	High that the apostrophe search caused a query error, because the access log and the error log agree on the time. Moderate that the flaw is exploitable, until an approved test or a code review shows how the input reaches the query.

The common error in a validation step tests something other than the original observation, such as checking that new code was deployed. A deployed fix that does not change the observed result has not been validated.

PART 4

Agents With Tools

An **agent** is a model that runs in a loop: it chooses a tool, calls it, reads the result, and chooses again, until it decides the task is done. Each tool call is an action in the world. An agent that can read files, fetch pages, and run commands can do an assessor's work at machine speed, and it can also do an attacker's work, including against the operator who started it.

The Lethal Trifecta

Simon Willison names the combination that makes an agent exploitable. The **lethal trifecta** is "access to your private data," "exposure to untrusted content," and "the ability to externally communicate in a way that could be used to steal your data." An agent with all three can be tricked "into accessing your private data and sending it to that attacker" [9]. The mechanism is the one Session 3 called indirect prompt injection: "LLMs follow instructions in content," and they are "unable to reliably distinguish the importance of instructions based on where they came from" [9].

Willison's practical advice is blunt. Vendor guardrails that claim to catch most attacks are not enough, because "in web application security 95% is very much a failing grade." For anyone combining tools, "the only way to stay safe there is to avoid that lethal trifecta combination entirely" [9]. He quotes a design-pattern paper whose summary states the constraint an assessment agent needs: "once an LLM agent has ingested untrusted input, it must be constrained so that it is impossible for that input to trigger any consequential actions" [9].

Trifecta property	Example in an assessment workflow
Private data	Captures and telemetry marked Confidential, configuration files, credentials found during testing
Untrusted content	Any response from a target, any web page fetched, any log field an outsider can write, such as a user agent or a username
External communication	Any tool that sends a request, runs a shell command, or posts a message

An assessment agent has all three by design: it reads target responses, holds assessment data, and sends traffic. Removing a leg is often impossible, so the defense is a gate the content cannot reach.

The OWASP Top 10 for Agentic Applications

The OWASP GenAI Security Project released its Top 10 for Agentic Applications on December 9, 2025, after review by what it calls "hundreds of experts" [10]. The ten risks are ASI01 Agent Goal Hijack, ASI02 Tool Misuse and Exploitation, ASI03 Identity and Privilege Abuse, ASI04 Agentic Supply Chain Vulnerabilities, ASI05 Unexpected Code Execution, ASI06 Memory and Context Poisoning, ASI07 Insecure Inter-Agent Communication, ASI08 Cascading Failures, ASI09 Human-Agent Trust Exploitation, and ASI10 Rogue Agents [10, 11]. ASI09 describes a risk this session cares about: "confident, polished explanations misled human operators into approving harmful actions" [10].

Two Agentic Risks in Full: ASI01 and ASI02

ASI01: Agent Goal Hijack

The OWASP document states the cause first: agents and the underlying model "cannot reliably distinguish instructions from related content." As a result, "attackers can manipulate an agent's objectives, task selection, or decision pathways" through "prompt-based manipulation, deceptive tool outputs, malicious artefacts, forged agent-to-agent messages, or poisoned external data" [11]. The entry separates goal hijack from a single bad answer. A prompt injection against a chatbot changes one response, and ASI01 "captures the broader agentic impact where manipulated inputs redirect goals" across multi-step behavior [11]. Its first example attack is EchoLeak, the Microsoft 365 Copilot case from Session 3, in which a crafted email made the assistant exfiltrate data with no user interaction [11].

Two of its mitigations map directly onto an assessment. The second is to "minimize the impact of goal hijacking by enforcing least privilege for agent tools and requiring human approval for high-impact or goal-changing actions." The fourth is to validate user intent and agent intent at run time, to require confirmation through human approval, a policy engine, or platform guardrails whenever the agent proposes actions that deviate from "the original task or scope," and to pause or block execution on any unexpected goal shift and record it for audit [11]. For an assessment agent, "the original task or scope" is the rules of engagement.

ASI02: Tool Misuse and Exploitation

ASI02 covers an agent that "operates within its authorized privileges but applies a legitimate tool in an unsafe or unintended way," for example "deleting valuable data, over-invoking costly APIs, or exfiltrating information" [11]. Its common examples include unvalidated input forwarding, where an "agent passes untrusted model output to a shell," and unsafe browsing, where a "research agent follows malicious links, downloads malware, or executes hidden prompts" [11]. One example attack fits a security operations setting closely. A security-automation agent receives an injected instruction and chains PowerShell, cURL, and internal APIs to exfiltrate logs. "Because every command is executed by trusted binaries under valid credentials," host monitoring "sees no malware or exploit, and the misuse goes undetected" [11]. The first mitigation is "least agency and least privilege for tools": per-tool profiles with scopes, rate limits, and egress allowlists, read-only database access where possible, and no send or delete rights for a summarizer [11].

Where the Gate Belongs in Code

Both entries point to the same design. The gate sits between the model's decision to call a tool and the tool's execution, and it is deterministic code, not another prompt. It reads the proposed call and checks it against the written scope before anything runs.

1. Parse every address, host name, and port from the proposed command, and refuse the call if any of them is outside the target list.
2. Refuse any prohibited technique by flag or tool name, whatever reason the model gives.
3. Hold any call that would change data, use a credential, or touch restricted information, and ask a named human for the four-part approval.
4. Log every proposed call, every verdict, and every approval, so the record shows what the agent tried as well as what it did.

A gate in code holds whether or not the model obeyed an instruction hidden in content, because the content never reaches the gate's logic. A gate written as a system prompt does not have that property.

GTG-1002: An AI-Orchestrated Intrusion and Its Human Gates

On November 13, 2025, Anthropic disclosed what it called "the first documented case of a large-scale cyberattack executed without substantial human intervention" [12]. The full report gives the detail [13].

Aspect	What the report states
Detection and attribution	Detected in mid-September 2025. Anthropic assesses "with high confidence" that a Chinese state-sponsored group it designates GTG-1002 conducted it (p. 3)
Scale and outcome	"The operation targeted roughly 30 entities and our investigation validated a handful of successful intrusions" (p. 3). Targets included technology corporations, financial institutions, chemical manufacturers, and government agencies (p. 8)
Architecture	A framework used Claude Code and open-standard Model Context Protocol (MCP) tools, and decomposed attacks into "discrete technical tasks for Claude sub-agents," each of which "appeared legitimate when evaluated in isolation" (p. 6)
Tools	"Overwhelmingly" open source penetration-testing tools: "network scanners, database exploitation frameworks, password crackers, and binary analysis suites," orchestrated through MCP servers (p. 12)
Getting past safeguards	"The key was role-play": the operators claimed to be employees of legitimate cybersecurity firms doing defensive testing (p. 8)
Division of labor	The AI executed "approximately 80 to 90 percent of all tactical work independently." Human effort was "estimated at 10 to 20 percent" (p. 7). The blog estimates "4-6 critical decision points per hacking campaign" [12]
Speed	"Peak activity included thousands of requests, representing sustained request rates of multiple operations per second" (p. 7)

Where the Humans Sat

The report places the human operators at "authorization decisions at critical escalation points": "approving progression from reconnaissance to active exploitation, authorizing use of harvested credentials for lateral movement, and making final decisions about data exfiltration scope and retention" (p. 7). In its example sequence, Claude spends one to four hours on discovery, analysis, and exploit development, and the operator spends two to ten minutes reviewing the findings and approving exploitation (p. 9). The structure is an approval gate before each consequential step, which is the structure of an authorized assessment. The difference is who signed the authorization and for which targets.

Over-Claiming

The report names a limitation that matters as much to defenders as to attackers: "Claude frequently overstated findings and occasionally fabricated data during autonomous operations, claiming to have obtained credentials that didn't work or identifying critical discoveries that proved to be publicly available information." The consequence was "requiring careful validation of all claimed results" (p. 4). The attackers needed the validation step for the same reason an assessor does. A model's report of a result is a claim about the result.

What the Case Does and Does Not Show

The case shows that an agent can carry most of the tactical work of an intrusion with commodity tools, and that its operators still placed human decisions at the consequential steps. It does not show the details of the targets or the defenses, which the report withholds. Anthropic also notes that it has visibility only into Claude usage, and that the case "likely reflects consistent patterns of behavior across frontier AI models" (p. 4). That last sentence is Anthropic's assessment, not a measurement.

PART 5

What the Measurements Say

Claims about AI in security operations range from "it cannot do anything real" to "it replaces the team." Three measurements bracket the question for this session: a public benchmark of offensive tasks, a model developer's own evaluation, and a randomized trial of AI assistance for defenders.

Cybench

Cybench is a framework of "40 professional-level Capture the Flag (CTF) tasks from 4 distinct CTF competitions," each run in an environment where an agent can execute commands and observe the output [14]. Because many tasks were beyond the agents of the time, the authors added subtasks that break each task into steps. The paper's headline result uses the time the fastest human team took to solve each task, its **first solve time**. Without subtask guidance, the best agents "successfully solved complete tasks that took human teams up to 11 minutes to solve," while "the most difficult task took human teams 24 hours and 54 minutes to solve" [14]. The paper was published at ICLR 2025, and its fourth version is dated April 12, 2025 [14].

The Claude Sonnet 4.5 System Card

Anthropic's system card for Claude Sonnet 4.5, dated September 2025, reports cyber evaluations in section 5 [15]. The model ran in a Kali-based environment with tools such as pwntools, Metasploit, Ghidra, and tshark (section 5.1, p. 33). On Cybench, Anthropic used 37 of the 40 challenges and grouped them by first solve time into Easy (under 21 minutes), Medium (21 minutes to 1.5 hours), and Hard (over 1.5 hours) (section 5.2.2, p. 36). It reports "an almost step-like improvement on Cybench, leading to a performance increase of roughly 20 percentage points across different numbers of trials, breaking 80% success for 30 trials," with gains "largely concentrated on Medium and Hard challenges" (p. 38). The card also reports cyber ranges of 25 to 50 hosts built with Carnegie Mellon University, in which Sonnet 4.5 outperformed other models on asset comprehensiveness, and five further range environments in which "no model succeeded at acquiring any critical assets" (section 5.3.2, pp. 42 to 45).

Read together, the two sources describe a fast change. In the Cybench paper's evaluation, agents finished only tasks that human teams solved within minutes. A little over a year later, a developer reports a model clearing most of the benchmark when allowed many attempts, with gains on the harder tasks. The tasks that typical scanning and enumeration labs assign sit well inside that range. That is the reason Lab 8 redesigns labs around authorization, validation, and defense rather than around whether a student can produce the output.

Security Copilot Randomized Trials

The defender-side measurement is a set of randomized controlled trials by Microsoft researchers, published as a whitepaper in January 2024 [16]. Subjects worked in a Microsoft Defender XDR environment on incident summarization, script analysis, incident reporting, and guided response, with half of them given Copilot for Security.

Group	Reported result
149 security novices, tested October 2023	Copilot users got 35 percent more questions correct overall, and 43 percent more on remediation steps. Holding accuracy constant, they were 26 percent faster (pp. 9 to 11)
147 security professionals, tested December 2023 to January 2024	Copilot users were 7 percent more accurate overall and finished the overall tasks 22 percent faster. They were 26.3 percent slower on the guided response task when accuracy was held constant (pp. 4 to 7)

The authors state two limits. Their tasks were "closely linked to Copilot's capabilities," and their novices were "security rookies with minimal skills," while experienced analysts "have a higher baseline, which leaves less room to improve" (p. 9). The trial measured an assistant that answers questions inside a security tool, with a human doing the work. It did not measure an agent acting on its own. The whitepaper is a Microsoft publication about a Microsoft product.

PART 6

Purple-Team Translation

An offensive finding is half done when it is confirmed. **Purple-team translation** completes it: the team turns the observation into a detection idea, a remediation with an owner, a validation step that repeats the original observation, and a confidence statement. The result is a finding a defender can use the next morning.

Part	What it contains	A common weakness
Detection idea	A data source the organization already collects, a condition that matches the behavior, and the false positives the condition will also match	A condition with no named data source, or no false positives listed
Remediation	Each change, with the person or team who owns it	"Fix the server," with no owner
Validation step	The original observation repeated after the fix, with the result expected if the fix worked	"Rescan," with no expected result
Confidence	How well the evidence supports each part of the finding, stated separately for each	One confidence for everything

A detection idea is a hypothesis about telemetry. It is tested the way any hypothesis is tested, with lines that should match and lines that should not. A rule that matches every positive and also matches the assessor's own authorized test is a rule that will page someone during the next assessment. Listing the expected false positives before writing the rule is how a team finds those lines.

Mapping to ATT&CK, With Limits

Session 3's mapping rules apply. Map behavior to a technique only where the evidence describes the behavior, and label a weak match. Two techniques are likely candidates in a web staging assessment. T1190, Exploit Public-Facing Application, describes adversaries who "attempt to exploit a weakness in an Internet-facing host or system to initially access a network," where the weakness "can be a software bug, a temporary glitch, or a misconfiguration" [20]. An attempt that failed is still evidence of the technique being tried, and the mapping should say "attempted." T1552.001, Unsecured Credentials: Credentials In Files, describes adversaries who "search local file systems and remote file shares for files containing insecurely stored credentials," including "configuration files containing passwords for a system or service" and passwords extracted "from backups" [21]. The technique's description centers on file systems and shares, so a match to a file served by a web server is partial, and the mapping should say so.

Time Zones and Clocks

Every timestamp carries a zone, stated or not. A log in UTC and a report in a local zone can describe the same minute with different clock times, and near midnight with different dates. Eastern Standard Time, for example, is five hours behind UTC. The first check on any conflicting record is to put both times in one zone. The second check is the clocks themselves. Two sensors that record the same request at different times, after zone conversion, disagree because at least one clock is wrong. The tie-breaker is the record of how each device gets its time. A host synchronized to the campus time servers outranks a device whose time source has failed.

Sensor Placement and Address Translation

A network sensor records the packets on the link where it sits. A firewall that translates source addresses changes what sensors downstream of it can see. A sensor upstream records the client's original address, and a sensor downstream records the address the firewall substituted. Two sensors can therefore report one client as two different addresses, and both are accurate. The network documentation, not the logs, says which sensor sits where.

Reading Zeek and Suricata Records, and the False-Positive Check

Zeek

Zeek writes one log per protocol. In both `conn.log` and `http.log`, `ts` is the timestamp, which Zeek's own example records show as seconds since January 1, 1970, UTC with six decimal places, and `uid` is the "unique ID for the connection," which ties records about one connection together across logs [17, 18]. `id.orig_h` and `id.resp_h` are the originator and responder addresses. In `http.log`, `status_code` is the response code, `response_body_len` is the size of the response body, and `proxied` holds "all of the headers that may indicate if the request was proxied" [17]. A value in `proxied` records a header the client sent. It is not an address the sensor observed.

`conn.log` records how each connection ended in `conn_state`. Three values cover most scan traffic: `SF`, "normal establishment and termination"; `REJ`, "connection attempt rejected"; and `S0`, "connection attempt seen, no reply" [18]. A TCP connect scan against a host with one open port leaves one `SF` row for the open port and a `REJ` row for each closed port that answered with a reset.

Suricata EVE

Suricata writes one JSON object per line, with a `timestamp`, a `flow_id` that correlates events from one flow, and an `event_type` such as `alert` or `http` [19]. An alert record carries the rule's `signature_id` and an `action`, whose possible values are "allowed" and "blocked" [19]. An `allowed` alert from a sensor in detection mode means the signature matched and the traffic passed. It says nothing about how the server responded. The HTTP fields of the same flow, such as `status` and `length`, carry the response. Suricata can also add the value of an `X-Forwarded-For` header to its records, an option that is off by default in the documented configuration [19]. Like Zeek's `proxied` field, that value is what the client claimed.

The False-Positive Check on an Attempted Exploit

SP 800-115 defines a **false positive** as "an alert that incorrectly indicates that a vulnerability is present" [1]. An exploitation alert raises a related question: did the attempt succeed? A signature matches the request. Success depends on the response and on the software. Three records answer the question better than the alert.

1. The response to the request: its status code and size, from the HTTP log or the server's own access log.
2. The software actually installed on the target, from a package inventory.
3. Any later activity from the same client that success would produce.

An escalation that calls an attempt successful on the strength of the alert alone has made an inference. The reconciliation states which of the three records support it.

Four Checks for a Conflicting Record

Check	Question
Time zone	Which zone does each source report in, and how do you know?
Clock	For one request seen by two sources, do the converted times agree? If not, which device's time source is documented as reliable?
Sensor placement	Where does each sensor sit relative to the firewall, and which source address can it see?
False-positive alternative	What did the alert match, how did the target respond, and what is installed?

PART 7

Redesigning Legacy Labs

Most scanning and penetration-testing labs were written to teach students to produce output: run the scan, list the open ports, name the vulnerability. Part 5 explains why that performance is no longer a good measure of learning, because a model can produce the same output or explain it. The redesign keeps the tools and changes what the student is scored on: working inside an authorization, recording raw output, confirming claims with independent evidence, asking for approval, and translating the finding into defense.

The Canvas

Lab 8 uses a twelve-field canvas from the NITIC facilitator manual. The prework's Legacy Lab Inventory collected nine of the same fields for the lab as it exists today, so the redesign starts by copying each inventory field into the canvas and changing it.

Canvas field	What the redesign changes
Learner and prerequisite, observable performance	The performance becomes a verb an observer can see that a model cannot perform for the student: classify an action against the scope, record raw output, validate a claim
Scenario decision	A decision with two defensible answers, made by a named role, so that the evidence matters
Authorized assets and stop conditions	Written rules of engagement with all seven elements and the approval rule, and a read-back as the first scored act
AI role and permissions	What students may ask a model, which data tier may go to which model, and where the approval gate sits
Expected failure or misconception	At least one planted misconception, such as a misleading banner, a stale scan, a decoy host, or an out-of-scope neighbor
Independent validation, student evidence	Two checkpoints where the student confirms a claim with evidence the claim did not supply, and the records that show it
Instructor truth and hints, reset and fallback	The true state of every target, the detection path for the misconception, a reset that returns every target to a known state, and saved output for a range that is down
Accessibility path, workforce alignment	A route to the same performance for a student who cannot use the default interface, and the job task it maps to

Planting a Misconception a Model Cannot Resolve Alone

A planted misconception works when the evidence that resolves it sits in a place a model cannot reach unless the student puts it there. A misleading banner is resolved by protocol behavior or a package list. A stale scan is resolved by its date against the window. An out-of-scope neighbor is resolved by the target element. A student who pastes only the scan into a model gets an answer built on the misconception. A student who gathers the second piece of evidence gets the truth. The validation checkpoint scores the gathering.

Accessibility and the Low-Compute Path

Two paths keep the redesign usable everywhere. Saved output gives a student who cannot use a terminal, or a section whose range is down, the same evidence as text files a screen reader can read. A small local model, or saved model outputs when no model is available, keeps the hypothesis step on hardware a college already owns and keeps Confidential lab data off hosted services. The performance scored is the same on every path: the student classifies, records, validates, and translates.

Reviewing a Redesigned Lab

The Safety Review

Every redesigned lab answers two questions before students see it. Could a student following the canvas touch anything outside the authorized assets? Could a student complete the validation checkpoints by asking a model, without looking at the evidence? A "yes" to either is a design defect with a known fix: tighten the target element and the stop conditions for the first, and move the resolving evidence out of the model's reach for the second.

A Review Checklist for the Seven Requirements

Requirement	A reviewer can confirm it when
Authorization	The rules of engagement name every target and every out-of-scope neighbor, state the window with a time zone, and say who approves what. The read-back is scored before any tool runs
Known expected findings	The instructor's truth covers every target, and the planted misconception has a written detection path that uses evidence inside the lab
AI-use boundaries	Each data tier is matched to the models it may enter, the approval gate has a named place, and the handout says a model's suggestion is never approval
Validation checkpoints	Each of at least two checkpoints names the evidence that confirms the claim and where the student finds it
Defensive extension	The telemetry needed for the detection idea exists in the lab and is given to the student
Reset	A named procedure returns every target to a known state, with a check that it did
Troubleshooting	Three likely failures each have a move, and one of them is a student exceeding scope: stop, preserve logs and shell history, and debrief

Questions to Bring to a Colleague's Canvas

1. Which action in the lab would be illegal on a network the student does not own, and where does the lab say so?
2. Which claim in the lab is designed to be wrong, and which record shows it?
3. What does a student submit that a model could not have written without the evidence?
4. What does the lab teach a defender, and which telemetry teaches it?
5. Would a student who cannot see the screen reach the same performance?

REFERENCE

Glossary

Term	Meaning as used in this session
Agent	A model that runs in a loop, choosing and calling tools and reading their results until it decides a task is done.
Approval gate	The point where a consequential test waits until a named person who owns the risk approves it in writing. In an agent, deterministic code before tool execution.
Banner	Identifying text a service sends, such as a greeting or a `Server` header. A claim the service makes about itself.
Consequential test	A test whose effects reach beyond reading: it changes data, can disrupt a service, uses a credential, or exposes restricted information.
Detection idea	A data source, a condition, and the expected false positives for one behavior.
False positive	An alert that incorrectly indicates that a vulnerability is present (SP 800-115).
First solve time	In Cybench, the time the fastest human team took to solve a competition task.
Hypothesis	One explanation of raw output, stated so that evidence could confirm or rule it out.
Immediate notification	In Lab 7's rules, the duty to report evidence that a third party already accessed restricted data, without stopping in-scope work.
Independent evidence	Two pieces of evidence that each rest on their own observation.
KEV catalog	CISA's Known Exploited Vulnerabilities catalog, with a date added and an action due date for each entry.
Least invasive method	The test that demonstrates a finding with the smallest effect on the target and on restricted data.
Lethal trifecta	Private data, untrusted content, and external communication in one agent (Willison).
MCP	Model Context Protocol, an open standard for connecting models to tools.
Out-of-scope neighbor	A system that shares a network, domain, or switch with an authorized target and is not itself authorized.
Purple-team translation	Turning one offensive observation into a detection idea, a remediation with an owner, a validation step, and a confidence statement.
Raw output	What a tool printed, unedited, with tool, version, command, target, and time with zone.
Rules of engagement (ROE)	Detailed guidelines and constraints for executing a security test, established before it starts (SP 800-115).
Scope read-back	A team's signed restatement of each element of the rules of engagement in its own words.
Stop condition	An observable event that ends the work at once and requires notification.
Validation step	The original observation repeated after a fix, with the result expected if the fix worked.
Version detection	Matching a service's responses against stored probe patterns to name its product and version (Nmap `-sV`).

REFERENCE

References, 1 to 12

1	K. Scarfone, M. Souppaya, A. Cody, A. Orebaugh, NIST Special Publication 800-115, "Technical Guide to Information Security Testing and Assessment," September 2008: section 4.2, section 7.2, section 7.3, Appendix B (Rules of Engagement Template), and the Appendix C glossary. https://nvlpubs.nist.gov/nistpubs/Legacy/SP/nistspecialpublication800-115.pdf
2	G. Lyon, "Port Scanning Techniques," Nmap Reference Guide, entry for -sT (TCP connect scan). https://nmap.org/book/man-port-scanning-techniques.html
3	G. Lyon, "Service and Version Detection," Nmap Reference Guide. https://nmap.org/book/man-version-detection.html
4	Apache Software Foundation, "Apache HTTP Server 2.4 Vulnerabilities": CVE-2021-41773, fixed in 2.4.50, released Oct 4, 2021; CVE-2021-42013, fixed in 2.4.51, released Oct 7, 2021. https://httpd.apache.org/security/vulnerabilities_24.html
5	NIST National Vulnerability Database, CVE-2021-41773, published Oct 5, 2021; CVSS 3.1 base score 9.8 (NIST), CWE-22; CISA KEV fields: added Nov 3, 2021, action due Nov 17, 2021. Read through the NVD CVE API 2.0. https://nvd.nist.gov/vuln/detail/CVE-2021-41773
6	NIST National Vulnerability Database, CVE-2021-42013, published Oct 7, 2021; CVSS 3.1 base score 9.8 (NIST), CWE-22; CISA KEV fields: added Nov 3, 2021, action due Nov 17, 2021. Read through the NVD CVE API 2.0. https://nvd.nist.gov/vuln/detail/CVE-2021-42013
7	CISA, Known Exploited Vulnerabilities Catalog, JSON feed, catalog version 2026.11.10, entries CVE-2021-41773 and CVE-2021-42013 (date added Nov 3, 2021; due date Nov 17, 2021; known ransomware campaign use: Known). https://www.cisa.gov/sites/default/files/feeds/known_exploited_vulnerabilities.json
8	Office of the Director of National Intelligence, Intelligence Community Directive 203, "Analytic Standards," signed Jan 2, 2015, with technical amendments signed Jan 21, 2022 and later. Copy served from archive.dni.gov. https://www.dni.gov/files/documents/ICD/ICD-203.pdf
9	S. Willison, "The Lethal Trifecta for AI Agents: Private Data, Untrusted Content, and External Communication," Simon Willison's Weblog, June 16, 2025. https://simonwillison.net/2025/Jun/16/the-lethal-trifecta/
10	OWASP GenAI Security Project, "OWASP Top 10 for Agentic Applications: The Benchmark for Agentic Security in the Age of Autonomous AI," blog post, Dec 9, 2025. https://genai.owasp.org/2025/12/09/owasp-top-10-for-agentic-applications-the-benchmark-for-agentic-security-in-the-age-of-autonomous-ai/
11	OWASP GenAI Security Project, Agentic Security Initiative, "OWASP Top 10 for Agentic Applications 2026," PDF, Dec 2025: ASI01 Agent Goal Hijack, pp. 8 to 10; ASI02 Tool Misuse and Exploitation, pp. 11 to 13. https://genai.owasp.org/resource/owasp-top-10-for-agentic-applications-for-2026/
12	Anthropic, "Disrupting the First Reported AI-Orchestrated Cyber Espionage Campaign," Nov 13, 2025, edited Nov 14, 2025. https://www.anthropic.com/news/disrupting-AI-espionage

REFERENCE

References, 13 to 21

13	Anthropic, "Disrupting the First Reported AI-Orchestrated Cyber Espionage Campaign," full report, November 2025, changelog entry Nov 17, 2025. Page numbers in the guide are the report's own. https://assets.anthropic.com/m/ec212e6566a0d47/original/Disrupting-the-first-reported-AI-orchestrated-cyber-espionage-campaign.pdf
14	A. K. Zhang et al., "Cybench: A Framework for Evaluating Cybersecurity Capabilities and Risks of Language Models," ICLR 2025; arXiv:2408.08926, submitted Aug 15, 2024, v4 Apr 12, 2025. https://arxiv.org/abs/2408.08926
15	Anthropic, "System Card: Claude Sonnet 4.5," September 2025, changelog through Dec 3, 2025: section 5, Cyber capabilities, pp. 32 to 44. https://assets.anthropic.com/m/12f214efcc2f457a/original/Claude-Sonnet-4-5-System-Card.pdf
16	B. Edelman, J. Bono, S. Peng, R. Rodriguez, S. Ho, "Randomized Controlled Trial for Copilot for Security," Microsoft whitepaper, January 2024. Copy opened at a Microsoft reseller's site because SSRN 4648700 blocked automated access on Nov 13, 2026. https://www.softcat.com/8817/1153/6619/Microsoft-Copilot-for-Security-productivity-findings-Whitepaper-Jan2024.pdf
17	The Zeek Project, "http.log," Book of Zeek (version 9.1.0-dev), page last updated Nov 13, 2026; and "base/protocols/http/main.zeek" for the field definitions, including ts and proxied. https://docs.zeek.org/en/master/logs/http.html
18	The Zeek Project, "base/protocols/conn/main.zeek," Book of Zeek (version 9.1.0-dev), conn_state values. https://docs.zeek.org/en/master/scripts/base/protocols/conn/main.zeek.html
19	Open Information Security Foundation, "Eve JSON Format" and "Eve JSON Output," Suricata 9.0.0-dev documentation, including the alert action field and the X-Forwarded-For (xff) option. https://docs.suricata.io/en/latest/output/eve/eve-json-format.html
20	MITRE ATT&CK Enterprise, T1190, Exploit Public-Facing Application, version 2.8, last modified May 12, 2026. https://attack.mitre.org/techniques/T1190/
21	MITRE ATT&CK Enterprise, T1552.001, Unsecured Credentials: Credentials In Files, version 1.3, last modified Sep 11, 2026. https://attack.mitre.org/techniques/T1552/001/

All URLs were opened on November 13, 2026, and every figure and quotation in this guide was read from the copy fetched that day. PDFs were read with `pdftotext`. The NVD records were read through the NVD CVE API 2.0. The ICD 203 address redirects to archive.dni.gov. The Security Copilot whitepaper was read from a Microsoft reseller's copy because SSRN refused automated access. ATT&CK technique pages and the Zeek and Suricata documentation change between releases. The versions cited are the ones current on November 13, 2026.

This material is based upon work supported by the U.S. National Science Foundation under Grant No. 2300188. Any opinions, findings, and conclusions or recommendations expressed in this material are those of the author(s) and do not necessarily reflect the views of the National Science Foundation.